

HTTP 轮询、SSE 和 WebSocket 的 IoT 的 Web 管理系统的研究和应用

龙举兵 罗忠巍

(东软集团股份有限公司, 辽宁 沈阳 110179)

✉ longjubing@neusoft.com; luozhw@neusoft.com



摘要:物联网(IoT)Web 管理系统中,不同应用场景对数据的时效性要求存在显著差异,而现有单一通信方案难以同时满足低延迟交互和资源优化的需求。针对这一问题,提出了一种基于 HTTP 轮询、服务器推送事件(SSE)和 WebSocket 的混合通信架构。该架构根据业务对时效性的不同要求,动态选择最适合的实时通信方式,综合考虑时效等关联因素,有效解决了 IoT Web 管理系统中的差异化通信需求,为大规模 IoT 设备的 Web 管理提供了可扩展的技术解决方案。

关键词:IoT; HTTP 轮询; 服务器推送事件; WebSocket; 混合通信架构; 实时通信

中图分类号:TP309.1 **文献标志码:**A

Research and Application of IoT Web Management Systems Using HTTP Polling, SSE, and WebSocket

LONG Jubing, LUO Zhongwei

(Neusoft Corporation, Shenyang 110179, China)

✉ longjubing@neusoft.com; luozhw@neusoft.com

Abstract: In Internet of Things (IoT) Web management systems, different application scenarios have significantly different requirements for data timeliness, while existing single communication schemes struggle to simultaneously meet the demands for low-latency interaction and resource optimization. To address this issue, this paper proposes a hybrid communication architecture based on HTTP polling, Server-Sent Events (SSE), and WebSocket. This architecture dynamically selects the most appropriate real-time communication method according to different business requirements for timeliness, comprehensively considers timeliness and other related factors, effectively resolves the differentiated communication needs in IoT Web management systems, and provides a scalable technical solution for the web management of large-scale IoT devices.

Key words: IoT; HTTP polling; Server Sent Events; WebSocket; hybrid communication architecture; real-time communication

0 引言(Introduction)

随着物联网(Internet of Things, IoT)^[1]技术的发展和应
用,智能设备数量呈爆炸式增长。据国际数据公司(IDC)预测,
到 2029 年的终端用户硬件、软件、预配置服务和专业服务领域
的潜在市场规模(Total Addressable Market, TAM)将超过

9520 亿美元^[2]。在这个背景下,探索和研究实时通信技术在
IoT 的 Web 系统中的应用有巨大的经济和社会效益。

当前主流的 Web 实时通信技术主要包括 HTTP 轮询、服
务器推送事件(Server-Sent Events, SSE)和 WebSocket,各自具
有不同的技术特点和适用场景。HTTP 轮询通过客户端定期

请求获取数据,实现简单但存在延迟高、网络开销大、服务器负荷高的问题;SSE 基于 HTTP 协议,支持服务器主动推送,具有自动重连特性,适用于单向数据通知场景;WebSocket 作为 HTML5 标准组件,通过持久的全双工连接,实现低延迟、高效率的双向通信,特别适合频繁交互的实时应用。

IoT Web 管理系统中,不同数据类型和应用场景对实时性需求存在显著差异。设备配置更新可容忍秒级延迟,设备状态监控需要近实时展示,而紧急控制指令的传输则要求毫秒级的响应速度。这种差异化的需求特征使得单一技术方案难以满足系统的整体性能要求,需要根据具体应用场景的特点选择最适合的通信技术,或者将多种技术进行有机整合。因此,本文旨在通过深入的理论分析和实际案例验证,系统研究 HTTP 轮询、SSE 和 WebSocket 三种技术在 IoT Web 管理系统中的应用特点,探讨基于应用需求的技术选择策略,并提出多技术融合的系统架构设计方案,为构建高效、可扩展的 IoT Web 管理系统提供理论指导和实践参考。

1 HTTP 轮询(HTTP Polling)

HTTP 轮询是一种通过基于 HTTP^[3] 的客户端定期向服务器发送请求来获取实时数据的通信技术,分为 Web 客户端和服务端^[4],采用传统的请求-响应模式。作为最早期的实时数据获取解决方案,HTTP 轮询以其简单直接的实现方式,在对实时性要求不高的应用场景中得到广泛应用。根据实现机制的不同,HTTP 轮询分为短轮询和长轮询两种类型。

1.1 短轮询技术

1.1.1 工作机制

短轮询的核心工作机制如图 1 所示,它的处理流程包括以下步骤:

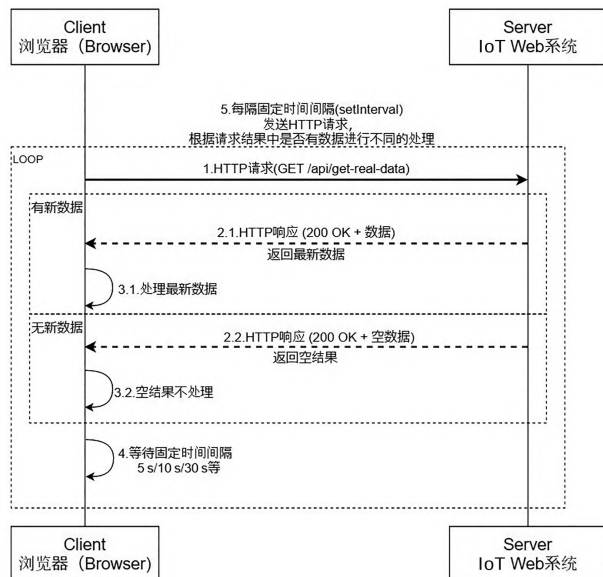


图 1 短轮询处理流程

Fig.1 Short polling processing flow

(1)发起请求:客户端发送 HTTP 请求。

(2)即时响应:服务器立即响应请求,返回当前的数据状态(无论是否有新数据)。

(3)数据处理:客户端接收并处理响应数据,包括有数据和无数据两种情况。

(4)间隔等待:客户端等待预设的固定时间间隔(如 5、10 或 30 s)。

(5)循环执行:重复执行上述流程,形成周期性轮询。

1.1.2 技术优势

实现简单:基于标准 HTTP,如果服务端已具备数据获取 API,通常只需修改客户端逻辑,服务器端无需额外改动。兼容性强:支持所有主流浏览器和网络环境,具有良好的通用性。

1.1.3 技术局限

资源消耗高:频繁的 HTTP 请求会增加服务器负载并占用大量网络带宽。实时性受限:数据更新存在固有延迟,总延迟包括轮询间隔时间、HTTP 请求-响应时间以及界面数据更新处理时间。

1.1.4 应用场景

短轮询技术主要适用于对数据时效性要求相对宽松的应用场景,如设备属性状态的定期更新,设备上报的非关键性数据采集,系统监控信息的周期性刷新。

1.2 长轮询技术

长轮询(Long Polling)是一种改进的轮询技术,通过延长 HTTP 请求的保持时间来实现准实时的数据推送。

1.2.1 工作机制

长轮询的核心工作机制如图 2 所示,它的处理流程包括以下步骤:

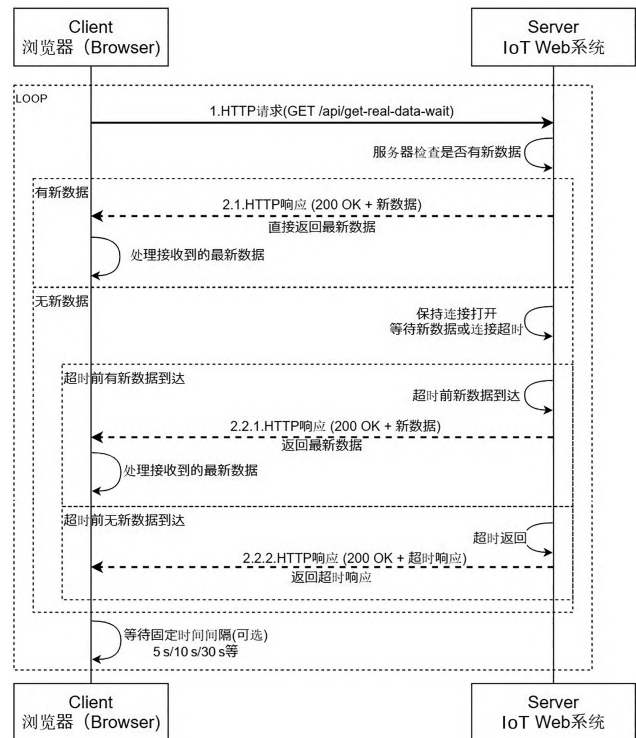


图 2 长轮询处理流程

Fig.2 Long polling processing flow

(1)客户端发起请求:客户端向服务器发送 HTTP 请求,请求获取最新数据。

(2)服务器智能响应:服务器接收到请求后,检查是否有新数据可供返回。

(3)若存在新数据,立即将数据返回给客户端。

(4)若暂无新数据,服务器保持连接处于等待状态,直到有新数据产生或连接时间达到预设超时时间。

(5)循环处理:客户端收到响应后,等待一段时间间隔或者立即发起下一次请求,形成持续的数据获取循环。

1.2.2 技术优势

相比传统的短轮询技术,长轮询具有以下显著优势:

(1)减少无效请求:避免频繁的空响应请求,大幅降低了无意义的网络交互。

(2)准实时通信:当服务器有新数据时能够立即推送给客户端,响应延迟显著降低。

(3)资源优化:显著降低了服务器负载和网络带宽消耗,提高了系统整体性能。实现简单:基于标准 HTTP,无需额外的协议支持,兼容性良好。

1.2.3 技术局限

(1)阻塞在服务器端,会大量占用服务区资源。

(2)不支持大量并发,并发过大会加大服务器压力,严重的会导致服务器崩溃。

1.2.4 应用场景

长轮询技术在以下场景中具有广泛的应用价值:

(1)物联网设备监控:设备状态变化的实时通知,支持设备异常告警和状态同步。

(2)金融数据服务:股票价格、汇率等金融数据的实时更新推送。

(3)在线游戏:游戏状态同步、玩家操作响应等实时性要求较高的场景。

2 服务器推送事件(Server-Sent Events)

服务器推送事件(Server-Sent Events)是一种基于 HTTP 的服务器推送技术,允许服务器主动向客户端推送数据,无需客户端频繁轮询^[5]。

2.1 工作机制

SSE 核心工作机制如图 3 所示,它的处理流程包括以下步骤:

(1)建立连接:客户端发送 HTTP 请求,在请求头中设置 Accept: text/event-stream。

(2)服务器响应:服务器接收请求后建立持久连接,返回响应头 Content-Type: text/event-stream。

(3)数据生成:服务器端持续监听事件源,实时生成或获取需要推送的数据。

(4)数据推送:服务器通过已建立的连接向客户端推送格式化的事件数据。

(5)客户端处理:客户端持续接收服务器推送的数据流,解

析事件并更新用户界面。

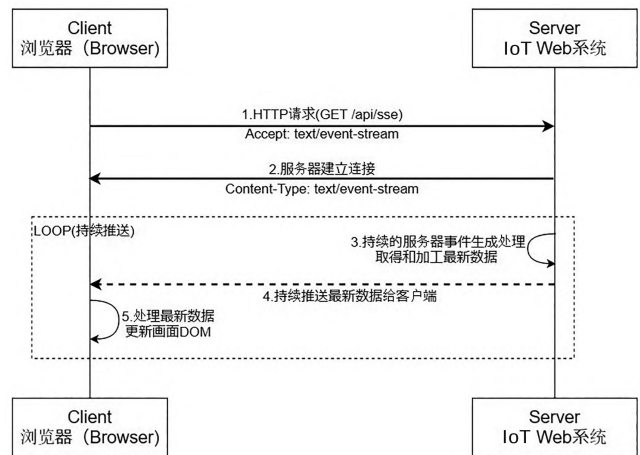


图 3 SSE 处理流程

Fig. 3 SSE processing flow

2.2 技术优势

(1)实现简单:基于标准 HTTP 协议,无需额外的协议支持。

(2)原生支持:现代浏览器原生支持 EventSource API,开发便捷。

(3)自动重连:连接断开时可自动重新建立连接,提高系统稳定性。

(4)资源高效:相比轮询方式,减少了不必要的 HTTP 请求开销。

(5)实时性好:服务器可即时推送数据,满足实时通信需求。

2.3 技术局限

(1)单向通信:仅支持服务器向客户端推送数据。

(2)一些老旧浏览器不支持,如 IE 等。

2.4 应用场景

SSE 特别适用于以下单向数据推送场景:

(1)实时通知系统:系统消息、用户通知的即时推送。

(2)金融数据推送:股票价格、汇率变化等实时数据更新。

(3)内容更新:新闻推送、社交媒体动态更新。

(4)监控面板:IoT 设备状态、系统性能指标的实时展示。

3 WebSocket

WebSocket^[6]是一种基于 TCP 协议的通信协议,支持服务器和客户端之间的双向通信,目标是为基于浏览器的应用提供一种双向通信的机制,不依赖于打开多个 HTTP 连接。与传统的 HTTP 请求-响应模式不同,WebSocket 在客户端和服务端之间建立持久连接,实现了真正的双向实时通信。

3.1 工作机制

WebSocket 的建立过程采用 HTTP 握手机制,随后升级为 WebSocket 协议进行通信。具体流程如图 4 所示,核心步骤

包括:

(1)客户端升级请求:客户端向服务器发送 HTTP 升级请求,请求头中包含 Upgrade: websocket 和 Connection: Upgrade 字段。

(2)服务器端接受协议升级:服务器验证请求后,返回 HTTP 101 状态码,确认协议升级(HTTP 状态码 101: 转换协议,Switching Protocols)^[7]。

(3)连接建立:握手成功后,TCP 连接保持开放状态,双方可随时发送和接收数据帧,连接建立后,客户端和服务端可以直接交换数据帧,无需额外的 HTTP 头信息,大幅降低了通信开销。

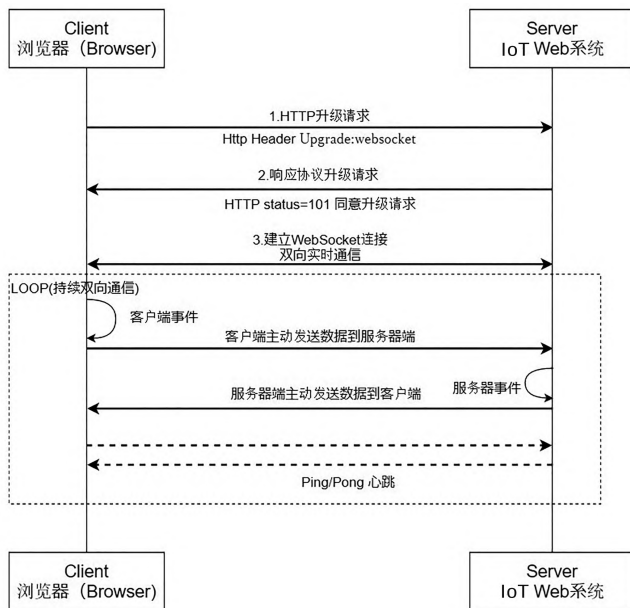


图 4 WebSocket 处理流程

Fig. 4 WebSocket processing flow

3.2 技术优势

WebSocket 相比传统 HTTP 轮询方式具有以下显著优势:

(1)实时性强:建立持久连接后,数据传输延迟极低,能够满足 IoT 系统对实时性的严格要求。

(2)通信效率高:避免了 HTTP 协议的冗余头信息,数据帧结构简洁,有效减少网络带宽消耗。

(3)双向主动推送:服务器可主动向客户端推送数据,无需客户端轮询,显著提升了系统响应效率。

(4)连接复用:单个连接可处理多种类型的数据交换,减少了连接建立和维护的系统开销。

(5)跨域支持:原生支持跨域通信,简化了分布式 IoT 系统的架构设计。

3.3 技术局限

尽管 WebSocket 具有诸多优势,但在实际应用中也存在一些不足:

(1)连接维护复杂:长连接需要处理网络中断、超时重连等异常情况,增加了系统复杂度。

(2)服务器资源占用:每个连接需要占用服务器内存和文件描述符,高并发场景下对服务器性能要求较高。

(3)防火墙和代理兼容性:部分网络环境下的防火墙或代理服务器可能不支持 WebSocket 协议。

(4)状态保持难度:需要在应用层实现心跳检测、连接状态管理等机制,开发复杂度相对较高。

(5)调试和监控困难:相比 HTTP 请求,WebSocket 连接的调试和性能监控需要专门的工具支持。

3.4 应用场景

WebSocket 的应用场景如下:设备状态实时监控,传感器数据流传输,远程控制指令下发,系统告警推送等。

4 应用案例(Application Instance)

为方便了解 3 种方案在实际项目中的应用,使用 HTTP 轮询、SSE 和 WebSocket 分别进行物联网设备遥测数据实时获取。为了便于对比,3 个方案实现的场景相同、技术栈相同(前端为 H5、CSS3、JS、Jquery3。后端为 SpringBoot3)。

场景:一栋大楼有很多实验室,每个实验室都安装有烟雾探测器和喷水装置。监控中心设置在实验室对面的大楼上,监控中心人员可以通过望远镜和无人机实时观测到实验室的火情。实验室的烟雾探测器实时监测实验室状况,上报数据到 IoT 平台,监控中心实时从物联网平台获取设备的实时遥测数据。当收到某项数据异常时,触发报警,监控人员通过望远镜和无人机确认是否真有火情。如果真有火情。远程控制相关实验室的喷水装置打开;如果火情已经消失,远程关闭喷水装置。本文的案例使用监控中心大屏(浏览器)和 IoT 平台(Web 系统)之间的实时遥测数据交互。

4.1 方案 1:HTTP 短轮询的实现方案

基于 HTTP 短轮询的实现方案具体如图 5 所示。

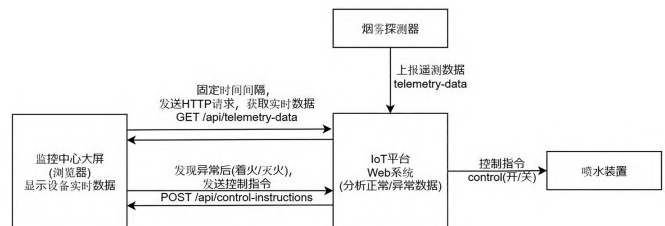


图 5 基于 HTTP 短轮询的方案

Fig. 5 HTTP short polling based solution

客户端定周期发送 HTTP 请求 (GET/api/telemetry-data) 取得实时遥测数据,并把取得的最新数据显示在页面上。服务器端接收到请求后,调用取得的最新数据处理,返回最新的数据,如果不存在最新数据,返回空数据。

当大屏显示报警数据的时候,监控人员确认火情,通过大屏发送控制命令,选择对应实验室的喷水装置,点击(关闭/打

开)按钮,触发控制指令下发请求(POST /api/control-instructions),等待指令调用结果返回,更新页面数据。服务器端接收到控制指令,调用 mqtt 连接,给对应的设备发送控制指令,返回指令执行结果给客户端。

核心代码(客户端):

```
setInterval(getDeviceRealData, 5000); // 每 5 s 取得一次数据
function getDeviceRealData() {
    $.ajax({ url: '/api/telemetry-data', // 数据请求 URL
        type: 'GET', dataType: 'json', timeout: 10 000, // 10 s 超时
        success: function (data, textStatus, xhr) {
            // 把最新的数据反映到页面中,显示实时数据(如温度、警告等)
            updateDeviceData(data);
        }
    });
}
```

4.2 方案 2: HTTP 长轮询的实现方案

实现方案具体如图 6 所示。

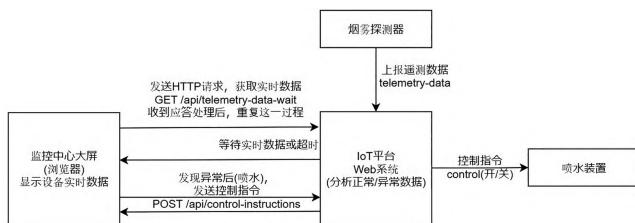


图 6 基于 HTTP 长轮询的方案

Fig. 6 HTTP long polling based solution

客户端发送 HTTP 请求(GET/api/telemetry-data-wait)获得最新的遥测数据,把数据显示在页面上后,重复这一过程。服务器端收到客户端的请求后,取得最新数据。如果存在最新的数据,则直接返回;如果不存在最新数据,休眠 3 s,继续调用取得遥测数据的处理后数据。反复这个过程直到取得最新数据或者请求耗时超过约定的超时时间。

核心代码(服务器端):

```
/** 服务器等待最新数据的超时时间: 25 min */
@Value("${http.wait.timeout:25}")
private int timeoutPeriodMinutes;

/** 阻塞等待取得最新的数据或者超时 */
@GetMapping("/api/telemetry-data-wait")
public List<DeviceDataDTO> getDeviceRealTimeWait() {
    List<DeviceDataDTO> realTimeList = null;
    // 取得最新数据
    realTimeList = dataManageService.getDeviceRealTimeData();
    // 超时时间(25 min < sessionTimeout 时间)
    long timeoutTimestamp = System.currentTimeMillis() +
        timeoutPeriodMinutes * 60 * 1000;
    long systemTimestamp = System.currentTimeMillis();
```

// 取到数据或者时间超时退出(如果没有取到实时数据,且请求没有超时)

```
while(realTimeList.size() == 0 && systemTimestamp <
    timeoutTimestamp) {
    Thread.sleep(3 * 1000);
    // 调用取得实时数据
    realTimeList = dataManageService.getDeviceRealTimeData();
    systemTimestamp = System.currentTimeMillis();
}
return realTimeList;
```

4.3 方案 3: 服务器推送事件的实现方案

实现方案具体如图 7 所示。

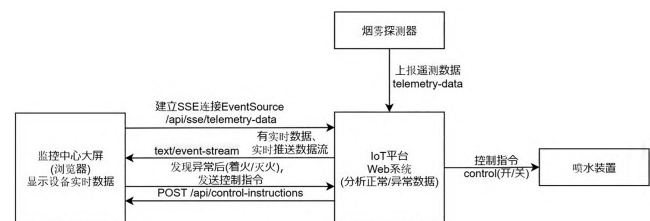


图 7 基于 SSE 的方案

Fig. 7 SSE based solution

客户端发送 HTTP 请求,建立 SSE 连接,并注册 SSE 消息处理函数(onmessage)来接收消息,把消息中的数据更新到页面中(使用 EventSource 实现^[8])。服务器端接收到连接请求后,取得设备实时数据。如果存在最新数据,则推送数据给客户端(text/event-stream);如果没有则休眠 3 s,重复这个处理。

核心代码(客户端):

```
// 初始化 SSE 连接
const eventSource = new EventSource('/api/event/telemetry-data');
// 监听服务器发送的消息
eventSource.onmessage = function (event) {
    const data = JSON.parse(event.data); // 解析服务器发送的数据
    updateDeviceData(data); // 更新页面内容
};
```

```
eventSource.onopen = function () { console.log('SSE 连接已建立'); };
eventSource.onerror = function () { // 监听连接错误事件
    console.error('SSE 连接发生错误'); eventSource.close();
};
```

核心代码(服务器端):

```
/** SSE 接口,用于向前端实时推送最新数据 */
@GetMapping(value="/api/event/telemetry-data", produces=
    MediaType.TEXT_EVENT_STREAM_VALUE)
public void sendTelemetryDataBySse(HttpServletResponse response) {
    // 设置响应头,确保 SSE 连接正常工作
    response.setContentType(MediaType.TEXT_EVENT_STREAM_
        VALUE);
    // 数据格式: SSE 发送的数据必须以 data: 开头,并以 \n\n 结尾。
```

```

final String eventDataFormat = "data: %s\n\n";
while (true) {
    List<DeviceDataDTO> list = dataService.
        getDeviceRealTimeData();
    // 如果没有数据,休眠 3s 后再次取得数据
    if (list.size() == 0) { Thread.sleep(3 * 1000); continue;}
    String data = objectMapper.writeValueAsString(list);
    // 把数据转换为 SSE 支持的数据格式
    String eventData = String.format(eventDataFormat, data);
    // 发送数据到前端
    response.getWriter().write("data: " + eventData + "\n\n");
    response.getWriter().flush();
}
}

```

4.4 方案 4: WebSocket 的实现方案

实现方案具体如图 8 所示。

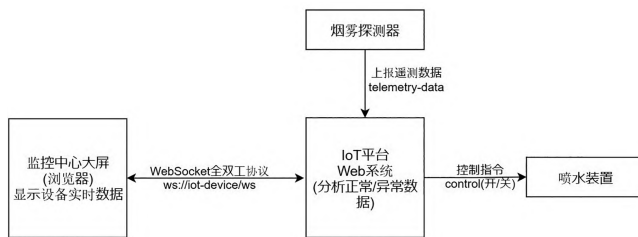


图 8 基于 WebSocket 的方案

Fig. 8 WebSocket based solution

客户端发送 WebSocket 连接请求 (ws://\$ {hostname:port}/api/websocket/telemetry-data), 并创建对应的消息接收的回调函数 (onmessage)。接收到消息后, 处理消息, 把消息内容更新到页面上 (使用 WebSocket 接口实现^[9])。服务器端接收到连接后, 管理所有的客户端连接会话, 并注册接收消息的处理函数和提供下发消息的接口。

核心代码(客户端):

```

// WebSocket 连接 URL
const wsUrl = 'ws://' + serverUrl + '/api/websocket/telemetry-data';
// 创建 WebSocket 连接
let socket;
function connectWebSocket() {
    socket = new WebSocket(wsUrl); // 创建新连接
    // 连接打开事件
    socket.onopen = function(event) { console.log('连接已建立'); };
    socket.onmessage = function(event) { // 收到消息的事件处理函数
        const data = JSON.parse(event.data);
        updateDeviceData(data); // 更新 UI 显示
    };
    // 连接关闭事件
    socket.onclose = function(event) { console.log(event); };
    // 连接错误事件

```

```

socket.onerror = function(error) { console.error(error); };
}

```

核心代码(服务器端):

```

/**配置类,配置 websocket 的监视 URI */
public class WebSocketConfig implements WebSocketConfigurer {
    /** 大屏使用的 websocket 处理类 */
    DeviceWebSocketHandler websocketHandler;

    public void registerWebSocketHandlers(WebSocketHandlerRegistry r) {
        r.addHandler(websocketHandler,
            "/api/websocket/telemetry-data").setAllowedOrigins("*");
    }
}

/**websocket 的通信管理类(管理连接/收发请求消息,处理请求) */
public class DeviceWebSocketHandler extends TextWebSocketHandler {
    /** 处理 websocket 接收到的消息 */
    DeviceWebSocketMessageHandler messageHandler;
    /** 处理客户端发送的消息 */
    protected void handleTextMessage(WebSocketSession wss,
        TextMessage tmsg) throws Exception {
        String payload = tmsg.getPayload();
        messageHandler.handler(payload);
    }
}

/** 向所有连接的客户端广播设备遥测数据 */
public void broadcastMessage(DeviceDataDTO data) {
    String msg = JsonUtil.convertToJson(data);
    sessions.forEach(s -> {
        if (s.isOpen()) {
            s.sendMessage(new TextMessage(msg));
        }
    });
}

/**向所有连接的客户端广播消息(要发送的消息) */
public void broadcastMessage(String strMsg) {
    sessions.forEach(s -> {
        if (s.isOpen()) {
            s.sendMessage(new TextMessage(strMsg));
        }
    });
}

/**向单个客户端发送消息(客户端会话,发送的消息) */
public void sendMessage(WebSocketSession wss, String strMsg) {
    if (wss.isOpen()) {
        wss.sendMessage(new TextMessage(strMsg));
    }
}
}

```

本案例综合考虑成本和技术实现难易度,选择长轮询来实现的服务器往客户端(大屏)主动通知的方式,大屏控制远端设备,使用 HTTP 发送 POST 请求给 IoT Web 系统,IoT Web 系统再通过 MQTT 协议下发控制指令给设备,执行相关的操作。一方面能够兼顾实时性的需求,一方面也能降低开发难度和成本。通过界面操作下发操作指令,符合用户操作习惯,用户操作明确且有实时反馈。如果对时效性要求特别高,且项目组的技术能够支撑 WebSocket,设备控制建议使用 WebSocket。案例系统的运行效果图如图 9 所示。



图 9 运行效果图

Fig. 9 Screen effect picture

5 结论(Conclusion)

本文对 IoT Web 管理系统中 3 种主要实时数据通信技术进行了系统性的研究与分析。通过理论阐述、性能对比和案例验证,得出以下主要结论。

首先,HTTP 轮询作为基于请求-响应模式的传统方案,具有实现简单、技术门槛低、兼容性强等优势,在对实时性要求相对宽松的应用场景中能够满足基本需求。该技术特别适合技术团队经验有限或优先考虑系统稳定性的项目实施,但它固有的高延迟、资源消耗大等缺点限制了在高实时性场景中的应用。

其次,服务器发送事件(SSE)基于 HTTP 协议实现了服务器向客户端的单向数据推送,相比轮询方式在数据传输的实时性和效率方面有显著提升。SSE 在以数据展示和状态监控为主的 IoT 应用中表现出明显优势,能够有效降低网络开销和服务

器负载,同时其自动重连机制增强了系统的可靠性。

第三,WebSocket 作为支持全双工通信的协议,能够实现客户端与服务器间的双向实时数据交换,在高实时性要求的 IoT 场景中表现卓越。以消防安全系统为例,火灾探测设备通过 WebSocket 实时上报险情数据至监控中心,同时监控中心能够即时下发控制指令实现设备联动响应,充分体现了该技术在关键任务场景中的应用价值。

通过对比分析发现,3 种技术方案各有其最适合的应用场景。HTTP 轮询适用于实时性要求不高的常规监控应用;SSE

适合服务器单向数据推送的展示类应用;WebSocket 则是双向实时交互应用的最佳选择。IoT 系统的设计者应当根据具体的业务需求、技术团队能力、系统复杂度以及成本预算等因素,综合选择最适合的数据通信方案,以实现系统性能与开发成本的最优平衡。

参考文献(References)

[1] 雷梦婷,薛亮.基于物联网的冷链运输管理信息系统设计[J].软件工程,2022,25(1):59-62.

[2] TORCHIA M, NARGAS M, SHRIVASTAVA A, et al. IDC's worldwide Internet of Things spending guide taxonomy[EB/OL]. (2025-06-01)[2025-07-10]. <http://my.idc.com/getdoc.jsp?containerId=US53463824>.

[3] Network Working Group. Hypertext transfer protocol—HTTP/1.1[S/OL]. (1999-06-01)[2025-07-10]. <https://www.w3.org/Protocols/rfc2616/rfc2616.html>.

[4] GOVRLE D, TOTTY B, SA ER M, et al. HTTP 权威指南[M].北京:人民邮电出版社出版,2020.

[5] WHATWG(网页超文本应用技术工作小组).HTML5 标准(9.2 Server-sent events)[S/OL].(2024-09-12)[2025-07-10].<https://html.whatwg.cn/dev/server-sent-events.html>.

[6] IETF(互联网工程任务组).The WebSocket Protocol[S/OL].(2021-12-01)[2025-07-10].<https://www.rfc-editor.org/rfc/rfc6455>.

[7] IETF(互联网工程任务组).Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content(6.Response Status Codes)[S/OL].(2024-06-12)[2025-07-10].<https://www.rfc-editor.org/rfc/rfc7231>.

[8] MDN contributors. Web technology for developers(Using server-sent events)[S/OL].(2025-05-16)[2025-07-10].https://jsschools.com/web_dev/server-sent-events-implementing-real-time-web-app/.

[9] WHATWG(网页超文本应用技术工作小组).WebSockets Living Standard (3. The WebSocket interface)[S/OL].(2024-01-24)[2025-07-10].<https://websockets.spec.whatwg.org>.

作者简介:

龙举兵(1985-),男(汉族),遂宁,工程师,本科.研究领域:技术赋能软件生产力的提升,IoT。

罗忠巍(1983-),男(汉族),长春,工程师,本科.研究领域:企业级软件高效开发和管理,IoT。本文通信作者。